

飞腾嵌入式 Ubuntu 系统
ROS2 软件使用手册
(V1.0)

Phytium 飞腾

飞腾信息技术有限公司
www.phytium.com.cn

2024 年 6 月 12 日

版权所有 © 飞腾信息技术有限公司 2024。保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 <https://service.phytium.com.cn/developer/>

目录

1 简介	1
2 准备工作	1
2.1 硬件环境	1
2.2 软件版本	1
2.3 前提设置	1
2.3.1 设置 locale	1
2.3.2 设置源	2
3 安装方法	2
3.1 软件包安装	2
3.1.1 apt 安装	2
3.1.2 运行环境	2
3.1.3 卸载流程	3
3.2 源代码安装	3
3.2.1 安装工具	3
3.2.2 下载源代码	3
3.2.3 安装依赖项	3
3.2.4 编译代码	4
3.2.5 运行环境	4
3.2.6 卸载流程	4
3.3 查看版本号	4
4 入门体验	5
4.1 控制小海龟	5
4.1.1 背景	5
4.1.2 安装 turtlesim	5
4.1.3 启动 turtlesim	5
4.1.4 操控 turtlesim	5
4.2 基本概念	5

4.2.1 节点	6
4.2.2 包	6
4.2.3 工作空间	6
4.2.4 节点和包和工作空间的关系	7
4.2.5 构建工具 Colon	7
4.3 hello world 示例	7
4.3.1 创建工作空间	7
4.3.2 创建包	7
4.3.3 创建节点	8
4.3.4 声明节点	10
4.3.5 Colcon 构建	10
4.3.6 运行环境	10
4.3.7 运行测试	11
5 常见问题	11
5.1 下载 ROS2 源代码失败	11
5.1.1 现象	11
5.1.2 原因	11
5.1.3 解决方案	12
5.2 更新 ROS 软件包依赖项信息失败	12
5.2.1 现象	12
5.2.2 原因	12
5.2.3 解决方案	12
6 参考文献	13
7 更新记录	13

1 简介

ROS 机器人操作系统是 Robot Operating System 的简称，其目前在业界被广泛用于快速搭建机器人系统。

ROS2 作为第二代机器人操作系统，不仅继承了第一代 ROS 的强大生态，而且采用了全新的架构设计，使得 ROS2 满足业界对于现代机器人系统的安全性、标准性、可靠性等多项要求。

关于 ROS2 更详细的介绍请参考官网 <https://docs.ros.org/>。

本文档介绍基于飞腾 D2000 开发板的飞腾嵌入式 Ubuntu22.04 系统上，安装使用 ROS2。

2 准备工作

2.1 硬件环境

ROS2 支持在飞腾 E2000 或者 D2000 开发板上安装。

飞腾开发板通过 DP 或者 HDMI 接显示器；

飞腾开发板通过 USB 集线器外接 USB 鼠标和 USB 键盘；

飞腾开发板通过集成网卡或者 PCIE 扩展网卡能够接入互联网。

2.2 软件版本

ROS2 版本是 Humble Hawksbill。

操作系统版本 Ubuntu 22.04。

2.3 前提设置

2.3.1 设置 locale

安装 ROS2 之前需要确保系统支持 UTF-8 语言环境，设置步骤如下所示。

```
$ locale
$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
$ locale
```

2.3.2 设置源

我们首先需要将 ROS2 apt 仓库添加到 Ubuntu 中。如下所示：

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
```

然后使用 apt 添加 ROS 2 GPG 密钥

```
$ sudo apt update && sudo apt install curl -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
```

接下来将软件源添加到源列表中

```
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

最后，在安装 ROS2 之前，再更新系统软件包

```
$ sudo apt update
```

到此为止，我们的准备工作全部完成，可以进行 ROS2 的安装工作了。

3 安装方法

ROS2 有两种安装方法，分别是软件包安装和源代码安装。官网推荐软件包安装，因为使用该方法安装 ROS2 需要操作的步骤很少。

3.1 软件包安装

3.1.1 apt 安装

安装 ROS2 humble

```
$ sudo apt install ros-humble-desktop
```

安装基础库，包括通信库，消息包，命令行等工具

```
$ sudo apt install ros-humble-ros-base
```

安装开发工具，包括编译器和其它用于构建 ROS 软件包工具

```
$ sudo apt install ros-dev-tools
```

3.1.2 运行环境

安装完成之后，ROS2 的安装包安装在 /opt/ros/humble/ 目录下，需要键入如下命令设置运行环境

```
$ source /opt/ros/humble/setup.bash
```

如果不想每次打开新的 shell 时都设置运行环境，可以通过如下操作将其添加到 shell 启动脚本

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

3.1.3 卸载流程

如果需要卸载 ROS 2 或在已从二进制文件安装，请键入如下命令

```
$ sudo apt remove ~nros-humble-* && sudo apt autoremove
```

删除存储库的流程如下

```
$ sudo rm /etc/apt/sources.list.d/ros2.list  
$ sudo apt update  
$ sudo apt autoremove  
$ sudo apt upgrade
```

3.2 源代码安装

3.2.1 安装工具

请在 Ubuntu22.04 中键入如下命令安装

```
$ sudo apt install -y \  
python3-flake8-docstrings \  
python3-pip \  
python3-pytest-cov \  
ros-dev-tools \  
python3-flake8-blind-except \  
python3-flake8-builtins \  
python3-flake8-class-newline \  
python3-flake8-comprehensions \  
python3-flake8-deprecated \  
python3-flake8-import-order \  
python3-flake8-quotes \  
python3-pytest-repeat \  
python3-pytest-rerunfailures
```

3.2.2 下载源代码

创建一个工作空间并克隆所有仓库

```
$ mkdir -p ~/ros2_humble/src  
$ cd ~/ros2_humble  
$ vcs import --input https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos src
```

其中最后一步经常执行失败，大家如果遇到该问题，请参考[常见问题 1](#)。

3.2.3 安装依赖项

注意：ROS 2 包是建立在频繁更新的 Ubuntu 系统上的。在安装新的包之前，始终建议确保当前的系统是最新的。

```
$ sudo apt upgrade
```

键入如下命令安装依赖项

```
$ sudo rosdep init
$ rosdep update
$ rosdep install --from-paths src --ignore-src -y --skip-keys "fastcdr rti-connext-dds-6.0.1
urdfdom_headers"
```

在执行 rosdep update 的时候经常执行失败，大家如果遇到该问题，请参考[常见问题 2](#)。

3.2.4 编译代码

```
$ cd ~/ros2_humble/
$ colcon build --symlink-install
```

3.2.5 运行环境

安装完成之后，ROS2 的安装包安装在~/ros2_humble/目录下，需要键入如下命令设置运行环境

```
$ . ~/ros2_humble/install/local_setup.bash
```

如果不想每次打开新的 shell 时都设置运行环境，可以通过如下操作将其添加到 shell 启动脚本

```
$ echo ". ~/ros2_humble/install/local_setup.bash" >> ~/.bashrc
```

3.2.6 卸载流程

```
$ rm -rf ~/ros2_humble
```

3.3 查看版本号

安装完毕之后，可以通过如下命令查看 ROS2 版本号

```
$ echo $ROS_DISTRO
humble
```

或者

```
$ printenv | grep ROS
ROS_VERSION=2
ROS_PYTHON_VERSION=3
ROS_LOCALHOST_ONLY=0
ROS_DISTRO=humble
```

4 入门体验

4.1 控制小海龟

4.1.1 背景

Turtlesim 是一个轻量级的用于入门级学习 ROS2 的模拟器。它以最基本的方式展示了 ROS2 的功能，我们通过下面章节的学习和操作，可以对 ROS2 的使用有直观的了解。

4.1.2 安装 turtlesim

```
$ sudo apt update
$ sudo apt install ros-humble-turtlesim
```

4.1.3 启动 turtlesim

要启动 turtlesim，请在图形界面的终端中输入以下命令：

```
$ ros2 run turtlesim turtlesim_node
```

这时，桌面会弹出一个窗口，窗口里面有一只小海龟。

4.1.4 操控 turtlesim

打开一个新的终端并再次启动 ROS2。

现在我们打开一个新的终端键入如下命令控制小海龟：

```
$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
'Q' to quit.
```

根据命令行提示信息，我们可以使用键盘的左右箭头分别控制小海龟逆时针和顺时针转动，使用上下箭头分别控制小海龟前进和后退。

体验了 ROS2 的小海龟模拟器之后，我们可能会产生疑问：上面两个命令分别代表什么含义？为什么可以控制小海龟运动？我们通过学习后续的章节可以找到答案。

4.2 基本概念

上一章节中，启动和操控 turtlesim 的命令格式为

```
$ ros2 run <package_name> <executable_name>
```

其中，第一个参数 run 表示启动，第二个参数 package_name 表示包，第三个参数表示节点

executable_name。所以，该命令的含义表示启动包 <package_name> 的节点 <executable_name>。

那么什么是包，什么是节点呢？我们继续学习。

4.2.1 节点

ROS2 节点是 ROS2 系统中的基本组成部分，负责执行特定的任务，并通过 ROS2 的通信机制与其他节点协作，以实现复杂的机器人行为和功能。通过将机器人的不同功能划分为不同的节点，可以实现更高的模块化、可重用性和可扩展性。

比如一个利用 ROS2 开发的机器人无人自动驾驶小车可以划分成几个节点。其中，一个节点负责控制车轮的前进和转动方向；一个节点负责通过雷达进行测距和定位，结合其它传感器提供环境感知能力；一个节点负责处理从雷达获取的数据等。

在控制小海龟的例子中，我们可以键入如下命令查看当前正在运行的节点。

```
$ ros2 node list
/teleop_turtle
/turtlesim
```

其中 turtlesim 表示第一个小海龟节点，teleop_turtle 表示第二个控制节点。

关于对节点更详细的介绍，请参考文献 2。

4.2.2 包

包是一个逻辑上独立的代码单元，它可以理解为一个容器，包含了一组相关的节点、服务、消息类型、配置文件等。每个包都可以独立地进行版本控制、编译和分发。包是 ROS2 代码组织的基本单位，通过将相关的代码和资源放在同一个包中，可以方便地进行代码管理和复用。例如，如果开发者希望能够安装代码或与他人共享代码，那么需要将代码组织成一个包，这样我们可以发布 ROS2 工作并允许其他人轻松构建和使用它。

安装包的方法如下

```
$ sudo apt install ros-<version>-package_name
```

4.1.2 章节我们就是通过安装获取到已经发布的 turtlesim 包，安装完毕后直接使用。

4.1.3 章节我们启动了一个名为 turtlesim_node 的节点，该节点是 turtlesim 包的一部分。

4.1.4 章节我们启动了一个名为 turt_teleop_key 节点，该节点也是 turtlesim 包的一部分。该节点允许用户通过键盘的箭头按键控制 turtlesim 模拟环境中的一个海龟的移动。

4.2.3 工作空间

工作空间可以理解成一个比包更大范围的容器。

一个典型的 ROS2 工作空间通常包含以下几个子空间：

src（代码空间）：用于存储所有包的代码、脚本等。这是开发人员进行主要编程工作的地方。

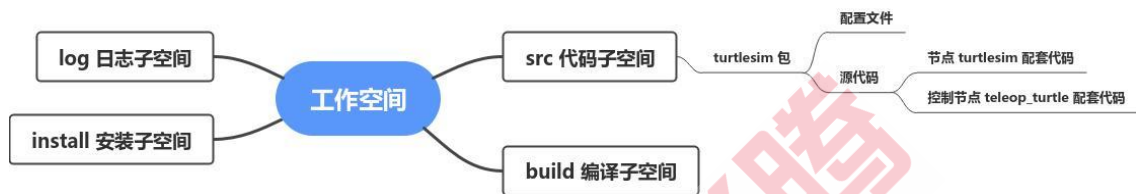
build（编译空间）：用于存储编译过程中产生的缓存和中间文件。这些文件是编译过程中生成的，通常不需要用户直接操作。

install（安装空间）：用于放置编译生成的可执行文件和脚本。当编译成功后，可执行文件将被放置在这个目录下，用户可以直接从这里运行它们。

log（日志空间）：用于保存编译和运行过程中产生的警告、报错等日志信息。这有助于开发人员调试和排查问题。

4.2.4 节点和包和工作空间的关系

节点，包和工作空间如下图所示：



4.2.5 构建工具 Colcon

上述章节我们讲述了节点，包和工作空间的基本概念，那么我们如何利用它们组织和编译我们的代码呢？那就是 ROS2 的构建工具 Colcon。

Colcon 是一个包构建工具，该工具可以在 ROS2 的工作空间创建和编译 ROS2 代码工程。

其安装方法如下：

```
$ sudo apt-get install python3-colcon-common-extensions
```

4.3 hello world 示例

4.3.1 创建工作空间

我们创建名称为 test_ws 的工作空间和 src 子空间，然后切换到 test_ws 工作空间的 src 子空间。

```
$ mkdir -p test_ws/src
$ cd test_ws/src
```

4.3.2 创建包

我们创建一个名称为 hello_py 的 python 版本的包。

```
$ ros2 pkg create hello_py --build-type ament_python --dependencies rclpy
```

创建包完成后的目录结构

```
test_ws/src$ tree
.
├── hello_py
└── hello_py
```

```
|   └── __init__.py
|── package.xml
|── resource
|   └── hello_py
|── setup.cfg
|── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py
```

4 directories, 8 files

4.3.3 创建节点

我们首先创建代码源文件

```
test_ws/src$ touch hello_py/hello_py/hello.py
```

```
test_ws/src$ tree
```

```
.
└── hello_py
    ├── hello_py
    │   └── hello.py
    │   └── __init__.py
    ├── package.xml
    ├── resource
    │   └── hello_py
    ├── setup.cfg
    ├── setup.py
    └── test
        ├── test_copyright.py
        ├── test_flake8.py
        └── test_pep257.py
```

4 directories, 9 files

我们开始编写代码创建节点，一般分为如下几个步骤

1. 导入必要的库
2. 创建主函数
3. 创建节点对象
4. 实现节点功能
5. 关闭 ROS2 环境并清理资源

我们按照该步骤编写。

```
test_ws/src$ cat hello_py/hello_py/hello.py
```

```
"""
```

导入必要的库

rclpy 是 ROS2 中用于 Python 语言的一个库，它提供了与 ROS2 通信的接口和工具。

我们可以使用它来创建 ROS2 节点、发布和订阅消息、调用服务等。这是 ROS2 中 Python 节点开发的基础。

```
"""
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
"""
```

创建主函数，用于节点的初始化、运行和清理过程

```
"""
```

```
def main(args=None):
```

```
    """
```

rclpy.init() 函数会处理与 ROS2 系统相关的初始化任务，以及解析任何与 ROS2 相关的命令行参数

```
    """
```

```
    rclpy.init(args=args)
```

```
    """
```

创建节点对象

创建一个新的 ROS2 节点，并将其命名为 "hello_node"。当使用 Node("hello_node") 创建一个节点时，我们可以在这个节点中定义发布者、订阅者、服务等，以便与其他节点进行通信。

```
    """
```

```
    node = Node("hello_node")
```

```
    """
```

从指定的 ROS2 节点（node）中获取一个日志记录器（logger），
并使用该日志记录器的 info 方法输出一条信息级别的日志消息，内容为 "Hello, World!"

```
    """
```

```
    node.get_logger().info("Hello, World!")
```

```
    """
```

spin 用于启动并运行 ROS 2 节点的事件循环。这个事件循环负责处理所有与节点相关的异步事件，例如接收和发布消息、处理服务请求和响应等。

当调用 rclpy.spin(node) 时，ROS2 节点会进入一个持续运行的循环，直到你显式地停止它（例如，通过按下 Ctrl+C 或者发送一个终止信号）。在这个循环中，节点会不断地监听和响应来自其他节点的消息和服务请求。

在大多数情况下，我们都会在 ROS2 节点程序的主函数（通常是 main 函数）的末尾调用 rclpy.spin(node)。

```
    """
```

```
    rclpy.spin(node)
```

```
    """
```

关闭 ROS2 环境，释放相关资源，并结束 ROS 2 节点的运行。

```
"""  
rcipy.shutdown()
```

4.3.4 声明节点

我们接着修改 setup.py 文件，该脚本的作用是定义如何构建、分发和安装 Python 包的入口点。

其中，蓝色的文字表示改动点。

```
test_ws/src$ cat hello_py/setup.py  
from setuptools import find_packages, setup  
package_name = 'hello_py'  
  
setup(  
    name=package_name,  
    version='0.0.0',  
    packages=find_packages(exclude=['test']),  
    data_files=[  
        ('share/ament_index/resource_index/packages',  
         ['resource/' + package_name]),  
        ('share/' + package_name, ['package.xml']),  
    ],  
    install_requires=['setuptools'],  
    zip_safe=True,  
    maintainer='user',  
    maintainer_email='user@todo.todo',  
    description='TODO: Package description',  
    license='TODO: License declaration',  
    tests_require=['pytest'],  
    entry_points={  
        'console_scripts': [  
            "hello_node = hello_py.hello:main"  
        ],  
    },  
)
```

entry_points 中的 'console_scripts' 部分允许我们定义从命令行直接调用的脚本。这表示当我们命令行运行 hello_node 时，实际上是在执行 hello_py.hello 模块中的 main 函数。

4.3.5 Colcon 构建

```
$ cd test_ws/  
test_ws$ ls  
src  
$ colcon build
```

4.3.6 运行环境

编译完毕后，工作空间目录多出了 3 个目录，即 4.2.3 章节阐述的 build（编译空间），install（安装空间）和 log（日志空间），如下蓝色部分所示。

```
test_ws$ ls
build install log src
```

设置新构建包的运行环境

```
test_ws$ source install/setup.bash
```

4.3.7 运行测试

键入如下命令运行节点，其中，hello_py 表示包的名字，hello_node 表示节点的名字。

```
test_ws/src$ ros2 run hello_py hello_node
[INFO] [1717483275.033209177] [hello_node]: Hello, World!
```

打开一个新的终端，设置运行环境并查看正在运行的节点

```
test_ws/src$ source install/setup.bash
test_ws/src$ ros2 node list
/hello_node
```

查找工作空间支持的所有的包是否包含名字有 hello 的包

```
test_ws/src$ ros2 pkg list | grep hello
hello_py
```

通过本章节的应用程序，我们可以创建单个节点。这在实际应用过程中是远远不够的。节点之间需要相互通信交换消息来满足具体的业务，比如 4.1 章节的控制节点通过通信控制小海龟节点旋转和前进后退。

5 常见问题

5.1 下载 ROS2 源代码失败

5.1.1 现象

用户在执行 3.2.3 章节下载 ROS2 源代码的时候有时会遇到下述问题。

```
$ vcs import --input https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos src
EEEEEEEEEEEEEEEE.....EE...E.....E.....E.....EEEEEEEEEE.EEE.EEEEEEEEE.....E.....
=== src/ament/ament_cmake (git) ===
Could not determine ref type of version: fatal: unable to access
'https://github.com/ament/ament_cmake.git': GnuTLS recv error (-110): The TLS connection was non-
properly terminated.
```

5.1.2 原因

下载代码需要花费较长的时间，但是 Ubuntu 终端无法稳定地连接到外国源网站。

5.1.3 解决方案

执行下述脚本，执行完毕即下载成功。

```
#!/bin/bash
ret=1
while [ $ret -eq 1 ]; do
    vcs import --input https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos src
    ret=$?
    echo "ret: $ret"
    sleep 10
done
```

5.2 更新 ROS 软件包依赖项信息失败

5.2.1 现象

用户在执行 3.2.3 章节下载 ROS2 源代码的时候有时会遇到下述问题。

```
$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
ERROR: error loading sources list:
The read operation timed out
```

5.2.2 原因

下载代码需要花费较长的时间，但是 Ubuntu 终端无法稳定地连接到外国源网站。

5.2.3 解决方案

执行下述脚本，执行完毕即下载成功。

```
#!/bin/bash
ret=1
while [ $ret -eq 1 ]; do
    rosdep update
    ret=$?
    echo "ret: $ret"
    sleep 10
done
```

[illegible]